CAPSTONE 31 — Research Report & Multi-AI Tool Synthesis

Target Audience: 2nd & 3rd-Year CSE Project Teams | Grounded in Pro Git & Official GitHub Docs

1. Target Audience Profile: Second- and Third-Year CSE Project Teams

Second- and third-year Computer Science & Engineering (CSE) undergraduate students transition from solo programming assignments (Data Structures, Algorithms, Object-Oriented Programming) to collaborative, multi-month team courses such as Software Engineering, Database Management Systems (DBMS), Operating Systems, and AI/ML mini-projects. Typical teams consist of 3 to 5 students working concurrently on a shared codebase under strict academic milestone deadlines.

Audience Dimension	2nd-Year CSE Teams (Semesters 3–4)	3rd-Year CSE Teams (Semesters 5–6)	Pedagogical & AI Primer Requirement
Prior Git Experience	Basic single-branch usage (`git add .`, `git commit`, `git push origin main`); frequent file sharing via ZIP or messaging apps.	Familiar with cloning and basic branches, but struggle with rebase vs. merge, conflict resolution, and structured Pull Requests.	Use concrete DAG pointer analogies and step-by-step terminal recovery recipes before introducing CI automation.
Primary Team Pain Points	Overwriting teammates' code on `main`, panic during merge conflicts, vague commit messages (*"update"*, *"final2"*).	"Works on my machine" integration failures night before demo, accidental commit of API keys/`.env` files, unreviewed PR merges.	Enforce branch protection rules, PR templates, automated commit-msg hooks, and GitHub Actions linting/testing.
AI Tool Usage Habits	Copy-pasting raw LLM code or Git commands without understanding flags like `--force` or `reset --hard`.	Generating GitHub Actions YAML workflows from LLMs that include deprecated action versions (@v2) or overly broad token permissions.	Train students on source verification against Pro Git and GitHub Docs.

2. Multi-AI Tool Research Synthesis (6 Free AI Tools Evaluated)

To answer the project question—*"How can AI help explain Git branching, pull requests, and simple CI pipelines to student teams?"*—five conversational/research AI systems plus GitHub's native tooling were queried and cross-evaluated across pedagogical clarity, command accuracy, YAML syntax currency, and security awareness.

AI Tool (Free Tier)	Primary Research & Project Role	Observed Strengths	Observed Limitations / Hallucinations
Google Gemini	Official documentation synthesis, quiz design, and CI security verification.	Accurately cited modern `git switch -c` syntax and current `actions/setup-python@v5` with `permissions: contents: read`.	Verbose explanations required prompt constraints to fit concise student cheat sheets.
ChatGPT (OpenAI)	Mental model analogies for branching, 3-way merges, and PR review etiquette.	Strong conceptual analogies ("parallel timeline sandbox" for topic branches; "quality airlock" for Pull Requests).	Initially suggested legacy `actions/checkout@v2` (Node 12, deprecated) until prompted to verify current versions.
Claude (Anthropic)	Drafting structured PR templates, Python unit tests, and Git hook scripts.	Clean, well-commented Python code for `commit-msg` regex validation and structured Markdown checklists.	Defaulted to `git checkout -b` in beginner tutorials unless explicitly asked for Git 2.23+ `git switch`.
DeepSeek	Static analysis edge cases, regex boundary testing, and CI failure debugging.	Strong reasoning on regular expression edge cases for Conventional Commits and `flake8` error codes (`E9,F63,F7,F82`).	Generated an unpinned third-party coverage action without warning about supply-chain SHA pinning.
Google NotebookLM	Source-grounded cross-checking against uploaded Pro Git chapters and GitHub Docs.	Zero extrinsic hallucinations when restricted strictly to uploaded official documentation excerpts; ideal for citation auditing.	Cannot execute code or test live Git hooks directly outside the uploaded corpus.
GitHub & GitHub Actions	Hosting sample repository, enforcing branch protection, and running CI pipelines.	Deterministic execution of `flake8` and `pytest` across Python 3.11 and 3.12 matrix runners on every Pull Request.	Requires explicit YAML configuration for least-privilege `GITHUB_TOKEN` permissions.

3. Comparative Analysis of Git Branching Workflows for Student Teams

According to Chapter 3.4 of the Pro Git Book (Branching Workflows) and the Atlassian Git Workflow Guide, branching strategies trade off structural ceremony against release velocity. Evaluating the three primary industry models demonstrates why **GitHub Flow** is the optimal architecture for 2nd- and 3rd-year CSE project teams.

Workflow Strategy	Branch Topology	Pros for CSE Student Teams	Cons / Failure Modes in Academic Projects	Verdict
GitHub Flow (GitHub Docs)	Single protected `main` branch + short-lived topic branches (`feat/*`, `fix/*`, `docs/*`) merged via PR + CI.	Minimal cognitive overhead; enforces peer review and automated CI before code touches `main`; keeps `main` demo-ready every week.	Requires disciplined automated tests so broken code does not slip into `main`.	**Recommended Standard** for 2nd & 3rd-year CSE teams.
Gitflow (Atlassian Guide)	Dual long-running branches (`main` + `develop`) plus `feature/*`, `release/*`, and `hotfix/*` branches.	Clear separation between tagged academic milestone releases and ongoing integration work.	Excessive merge ceremony for 4-person semester teams; students frequently merge into the wrong long-running branch and create complex divergence.	Over-engineered for semester courses; reserve for large multi-release systems.
Trunk-Based Development	Developers commit directly to `main` (or tiny branches lasting <1 day) backed by feature flags.	Eliminates long-lived merge conflicts and maximizes integration frequency.	High risk for undergraduates still learning testing; a single unreviewed syntax error breaks the entire team's build before lab evaluation.	Unsafe without mature feature-flagging and senior engineering discipline.

4. Verified Authoritative Sources & Grounding Ledger

Every technical explanation, command, YAML directive, and security rule in this project has been verified against official primary sources:

- **Pro Git Book (2nd Edition) by Scott Chacon & Ben Straub**: Chapter 3.2 Basic Branching and Merging and Chapter 3.4 Branching Workflows — verifies fast-forward vs. three-way merge mechanics and topic branch lifecycles.
- **GitHub Flow Official Documentation**: GitHub Docs — Following GitHub flow — verifies the 6-step branch, commit, push, PR, review/CI, merge, and branch-deletion cycle.
- **GitHub Protected Branches & Rulesets**: GitHub Docs — Managing a branch protection rule — verifies required status checks, required approving reviews, stale review dismissal, and blocking force-pushes.
- **GitHub Pull Request Templates**: GitHub Docs — Creating a pull request template — verifies placement in `.github/pull_request_template.md`.
- **GitHub Actions Python CI Guide**: GitHub Docs — Building and testing Python — verifies `actions/setup-python@v5`, `cache: 'pip'`, matrix testing, and `pytest`/linter integration.

- **GitHub Actions Security Hardening**: GitHub Docs — Secure use reference — verifies least-privilege `GITHUB_TOKEN` permissions (`contents: read`), secret redaction limits, script-injection mitigation via environment variables, and commit-SHA/tag pinning.
- **Conventional Commits Specification v1.0.0**: Conventional Commits v1.0.0 — verifies structured commit headers (`feat`, `fix`, `docs`, `test`, `ci`, `refactor`) for automated hook validation.

5. Executive Research Summary & Actionable Recommendations for CSE Teams

Synthesizing findings across all six AI tools and official documentation yields four core operational rules for second- and third-year CSE student project teams:

- **Adopt GitHub Flow Over Gitflow**: Maintain a single protected `main` branch and create short-lived topic branches (`feat/*`, `fix/*`) for every task. This eliminates the multi-branch merge confusion of Gitflow while keeping `main` ready for faculty lab demonstrations every week.
- **Use AI as a Tutor and Scaffolding Partner, Never an Unverified Oracle**: Conversational AI tools (ChatGPT, Gemini, Claude, DeepSeek) excel at explaining Git's Directed Acyclic Graph (DAG) pointer mechanics and drafting PR checklists, but frequently emit deprecated GitHub Actions tags (`@v2`) unless verified against GitHub Actions Documentation.
- **Automate Fast Feedback Both Locally and in the Cloud**: Combine a local client-side `commit-msg` hook (catching malformed commit headers and accidental API keys in <0.1 seconds) with a cloud GitHub Actions workflow running `flake8` and `pytest` across Python 3.11 and 3.12 on every Pull Request.
- **Enforce Technical and Ethical Governance in Repository Settings**: Configure branch protection rules requiring at least one peer review approval and passing CI status checks before merging, set `permissions: contents: read` on workflows, and credit all templates and AI prompts transparently.